PHENIKAA UNIVERSITY

SCHOOL OF COMPUTING

FINAL PROJECT REPORT

SUBJECT: SOFTWARE ARCHITECTURE

ONLINE STUDENT GRADES MANAGEMENT SYSTEM

(Online Student Grade Management System)

Instructor: Nguyen Quang Dung

Class: Software Architecture-1-3-25(N01)

The group of students who completed the project:

STT	Full name	Student ID
1	Nguyen Quoc Thai	23010225
2	Nguyen Hoang Son	23010100

Hanoi, June 2026

PREFACE

In the era of digital transformation, the application of information technology to education management is an inevitable trend. Managing grades – a core activity in training – still suffers from many shortcomings when done manually: lack of transparency, difficulty in controlling errors, and time-consuming report compilation. A well-designed software system not only solves these problems but also ensures scalability, maintainability, and the ability to meet stringent non-functional requirements.

This report presents the entire process of analysis, design, implementation, and testing of the Student Grade Management System (SGMS) – a final project for the Software Architecture course, carried out from April to June 2026 by a group of two students.

The report is organized into seven chapters, covering the problem, requirements analysis (SRS), architectural design (SDD) with Layered Architecture, C4 Model, ADR, implementation, testing, user guide, and conclusion. The entire presentation adheres to the structure and report writing guidelines of the textbook, while applying core concepts of modern software architecture: components, connectors, quality attribute scenarios, ASRs, Utility Tree, and key architectural patterns.

Hopefully, this report will be a useful reference for other student groups in their studies and in carrying out similar projects.

THANK YOU

To complete this Software Architecture final project and report, our team received invaluable guidance, support, and encouragement from many individuals and groups.

First and foremost, we would like to express our deepest gratitude to Mr. Nguyen Quang Dung, our lecturer, who has diligently taught and imparted fundamental and modern knowledge about software architecture, as well as directly guided us throughout the analysis, design, and implementation of the system. His discussions and feedback helped us better understand the importance of choosing the right architectural style, how to document using C4 Model, and how to record decisions using ADR.

We would also like to thank the Faculty of Information Technology – Phenikaa University for providing facilities, computer labs, and a wealth of reference materials.

Despite our best efforts, this report and product inevitably contain some shortcomings. We sincerely hope to receive feedback from our professors and classmates to further improve our future research.

Thank you very much!

INDEX

LIST OF FIGURES

LIST OF TABLES AND FIGURES

CHAPTER 1: PROJECT OVERVIEW

1.1. Stating the problem

Currently, the management of grades at many universities and colleges in Vietnam still has many shortcomings. The traditional method using Excel files or paper notebooks leads to serious problems: (1) grades can be modified without control, (2) students have difficulty in transparently checking their academic results, (3) lecturers spend a lot of time compiling, calculating average grades and preparing reports.

According to the "cost of fixing errors increases over time" principle pointed out by Bass, Clements & Kazman (2021), an error discovered during the operational phase can be 100 times more expensive than one discovered during the design phase. Therefore, building a software system with the correct architecture from the outset is a mandatory requirement.

The proposed Online Student Grade Management System (SGMS) aims to automate the entire process: grade entry, storage, average grade calculation, lookup, and report generation. The system ensures data integrity, transparency, and security while reducing administrative workload for faculty and the academic department.

1.2. Project Objectives

The project sets specific, measurable goals according to the SMART criteria:

1. The complete web application includes four core functionalities: authentication/authorization (3 roles: Student, Lecturer, Admin), student/course management, grade entry (both manual and CSV file upload), grade lookup, and PDF/Excel report generation.
2. Successfully implemented a 4-story Layered Architecture, adhering strictly to the principle that "the upper floor only communicates with the floor below," and documented architectural decisions using ADRs (Architecture Decision Records).
3. Ensure the system achieves an average response time of < 2 seconds for 95% of requests (P95) and an uptime of $\geq 99\%$ during business hours.
4. The product was completed in 10 weeks, with full documentation including SRS, SDD, a real-world demo, and source code delivered.

1.3. Project Scope

Within Scope:

- Responsive web application (React.js).
- JWT authentication system, role-based authorization (RBAC).
- CRUD: students, courses, instructor assignments.
- Enter the scores for each section (attendance, midterm, final) and upload a validated CSV file.
- Automatically calculates the final score using a 10-point scale and a letter grade system (A, B, C, D, F).
- Look up grades by student, by class, and export reports to PDF.
- Logging actions that change the entry point (audit trail).

Out of Scope:

- Native mobile application (iOS/Android).
- Integrate a tuition payment gateway.
- Online training (E-learning) or assignment management.
- Integrates with the school's training system (LMS, CMS) via API.

1.4. Technology Overview

Layer	Selection technology	Reasons for choosing
Presentation	React.js 18, TailwindCSS	Component-based, reusable, high-performance, large community.
Business Logic	Node.js 20 + Express.js 4	Lightweight, fast, non-blocking I/O, compatible with REST API, and uses the same language as the frontend.
Data Access	Prisma ORM (TypeScript)	Type-safe, automatic migration, minimizes SQL injection errors.
Database	PostgreSQL 15	Compliant with ACID, supports transactions, checks constraints, and is suitable for highly structured data.

DevOps & Tools	Docker, Git, GitHub Actions	Containerization of the environment, version management, basic CI/CD.
Diagramming	Draw.io, Structurizr DSL	Architectural documentation is done using the C4 Model, making it easy to edit and version control.

CHAPTER 2: REQUIREMENTS ANALYSIS (SRS)

2.1. Functional Requirements

ID	Function	Detailed description	Prioritize
FR01	Login & Permissions	Users log in using email/password; the system returns a JWT token; API access is granted based on role (admin, teacher, student).	High

FR02	Student Management (CRUD)	Admin/Lecturer can add, edit, and delete students (full name, student ID, class, course).	High
FR03	Course management & assignments	Admin creates courses (course code, course name, number of credits); assigns instructors to teach them.	High
FR04	Enter scores (manual and file upload)	The instructor enters the component scores (attendance, midterm, final) for each student; or uploads a CSV file (using a pre-made template).	High
FR05	Automatic overall score calculation	The system automatically calculates the weighted average score and letter grade; and updates the score sheet.	High
FR06	Check your scores	Students can view detailed grades for the courses they have taken; instructors can view grades for their classes; administrators can view all grades.	High

FR07	Export report to PDF/Excel	Export the grade sheet of a class or a student as a PDF (with a simulated digital signature) or Excel file.	Medium
FR08	Audit Log (Audit Change History)	Record all actions of editing/deleting points: who edited, when, and the old and new values.	Medium

2.2. Non-functional requirements – Quality Attribute Scenarios

Each non-functional request is described according to a 6-component structure (Stimulus Source → Stimulus → Artifact → Environment → Response → Response Measure).

Attributes	Scenario	Feedback measurement

Performance	200 students can check their grades simultaneously at the end of the semester (8:00 PM – 10:00 PM).	Average response time < 2 seconds (P95), through load test with k6.
Availability	The system operates during business hours (8 AM – 5 PM, Monday – Friday).	Uptime ≥ 99% (≤ 3.65 days downtime/year).
Security	Unauthenticated user attempts to call API/api/v1/grades; or an unauthorized user (student) attempts to alter the grade.	HTTP 401/403; no data points leaked; all requests are logged.
Integrity	The instructor entered the final grade as 12 (which is higher than 10).	The system refused to save, returning error 400 with the message "The score must be between 0 and 10".
Usability	The web interface must display well on devices with resolutions ranging from 375px (mobile) to 1920px (desktop).	No layout overflow, menu can be collapsed on mobile; meets responsive testing standards.
Maintainability	A new developer needs to add a new type of component score (e.g., project score).	Revision time is ≤ 2 working days, requiring only changes to the Business Layer and Database schema; the Presentation layer will not be affected.

2.3. Use Case Diagram and Specification

Overall Use Case Diagram

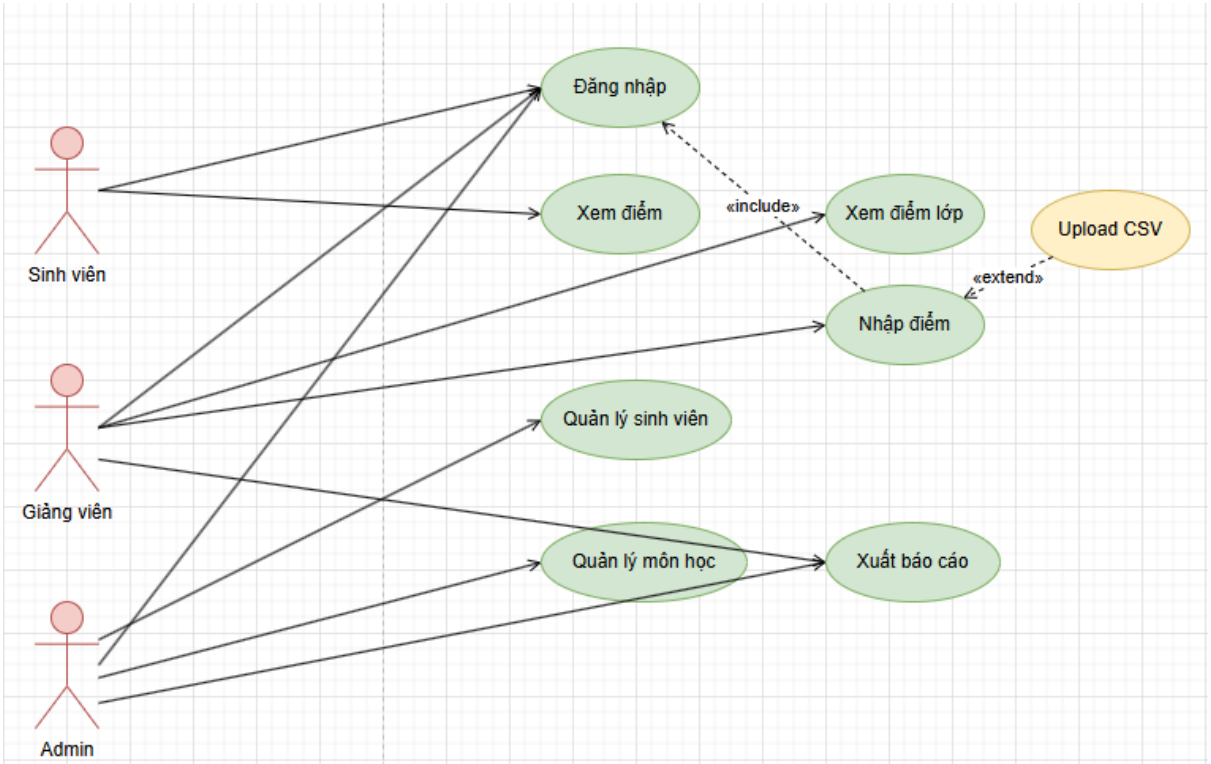

Figure 2.1:

The system has three main actors: Students, Lecturers, and Admins.

- Students: View grades, View summary transcript.
- Instructor: Enter grades (manually/upload), View class grades, Export reports.
- Admin: Manage students, manage courses, assign instructors, view all reports.
- The "Authenticity" use cases are included in all use cases that require login.
- The use case “Upload CSV file” is extended from “Enter grades” (when the instructor selects the upload option).

Detailed specifications of the three most important use cases (Table 2.3):

School	Content
UC-01	Log in
Agent	Students, Lecturers, Admins
Preconditions	The user already has an account (email + password) in the system.
Postconditions	The system generated a valid JWT token; the session was established.
Main stream	1. The user enters their email and password. 2. The system verifies the information. 3. The system returns a token and role information. 4. Redirects the user to the corresponding dashboard.
Alternative flow	2a. Incorrect email/password → displays the error "Login failed". 2b. Account locked → displays the error.

School	Content
UC-03	Enter scores (manually)
Agent	Lecturer
Preconditions	The instructor has logged in; the course has been assigned.
Postconditions	Scores are saved to the database; changes are logged; and the final score is updated (if all component scores are met).

Main stream	1. The instructor selects the class/subject. 2. The system displays the list of students. 3. The instructor enters the component scores for each student. 4. Click "Save Score". 5. The system validates (0-10). 6. Saves to the database and recalculates the average score. 7. A success message is displayed.
Alternative flow	5a. Invalid score → error message and request to re-enter. 4a. If not enough columns have been entered → error message.

(Similarly for UC-04 – Student Grade Lookup)

2.4. Architecturally Significant Requirements (ASRs)

2.4.1. Utility Tree – Quality Priority Tree

Based on hypothetical interviews with stakeholders (students, faculty, training department), we constructed a Utility Tree with 5 quality attributes, each attribute having 1-2 scenarios along with their importance level (Biz Value) and architectural impact level (Arch. Impact).

Quality Attribute	Refinement	ASR Scenario (situational scenario)	We are Val	Arch Imp
Security	Authentication & Authz	All APIs except /login require a valid JWT; tokens expire after 15 minutes; students cannot access the grade entry API.	H	H
Performanc e	Response time under load	At the end of the term, with 300 students simultaneously checking their grades, the /grades/my-grades API responded in an	H	H

		average of < 2 seconds (P95).		
Integrity	Data correctness	When instructors enter grades, the system must ensure that the grades are within [0,10]; if the uploaded CSV file contains an incorrect line, the entire file will be rejected (transaction).	H	H
Availability	Uptime	The system is available from 8 AM to 5 PM (business hours) with an uptime of ≥ 99%, and on the day the results are announced, the uptime must be ≥ 99.9% within the first 2 hours.	H	M

Maintainability	Adding new grade component	When the school requires an additional "Project" grade column (10%), the development team can complete it in ≤ 2 days without disrupting the existing logic.	M	H

2.4.2. ASR Table and Architectural Impacts

ID ASR	Describe	Architecture Driver

ASR-01	JWT token, lifetime of 15 minutes, token refresh every 7 days.	Use presentation-layer authentication middleware (Express middleware). Use httpOnly cookies for refresh tokens.
ASR-02	The API for point lookup must take less than 2 seconds with 300 concurrent users (CCU).	Apply caching to the score table (Redis) and pagination; optimize SQL statements (only select necessary columns).
ASR-03	Scores are only saved after validation; uploading the CSV file must be a transaction.	Use Prisma transactions in the Business Logic layer; if any line fails → rollback the entire process.
ASR-04	Guaranteed 99% uptime.	Implement a health check endpoint (GET /health) and use a process manager (PM2) or a Docker restart policy.
ASR-05	It's easy to add a new score column.	Flexible database design: tables`grade_components`The business layer uses the Strategy Pattern to calculate the final score.

(The link between ASR and ADR will be discussed in detail in Chapter 3.)

CHAPTER 3: ARCHITECTURAL & SYSTEMS DESIGN (SDD)

3.1. Choosing an architectural style

The project has chosen a Layered Architecture style with 4 floors:

1. Presentation Layer: React.js SPA.
2. Business Logic Layer: Node.js + Express.
3. Data Access Layer: Prisma ORM.
4. Database Layer: PostgreSQL.

Reason for selection (recorded in ADR-001):

- Suitable for traditional, small-scale CRUD applications (2 members).
- Easy to understand, and each layer can be tested independently.
- Clearly separate concerns.
- No complexity of Microservices or Event-Driven Architecture is required.

The options considered:

- Microservices: Overkill, increases operating costs, unsuitable for small teams.
- Modular Monolith (Server-side MVC): Less flexible when wanting to separate frontend/backend; does not take advantage of the React ecosystem.

3.2. Overall System Architecture

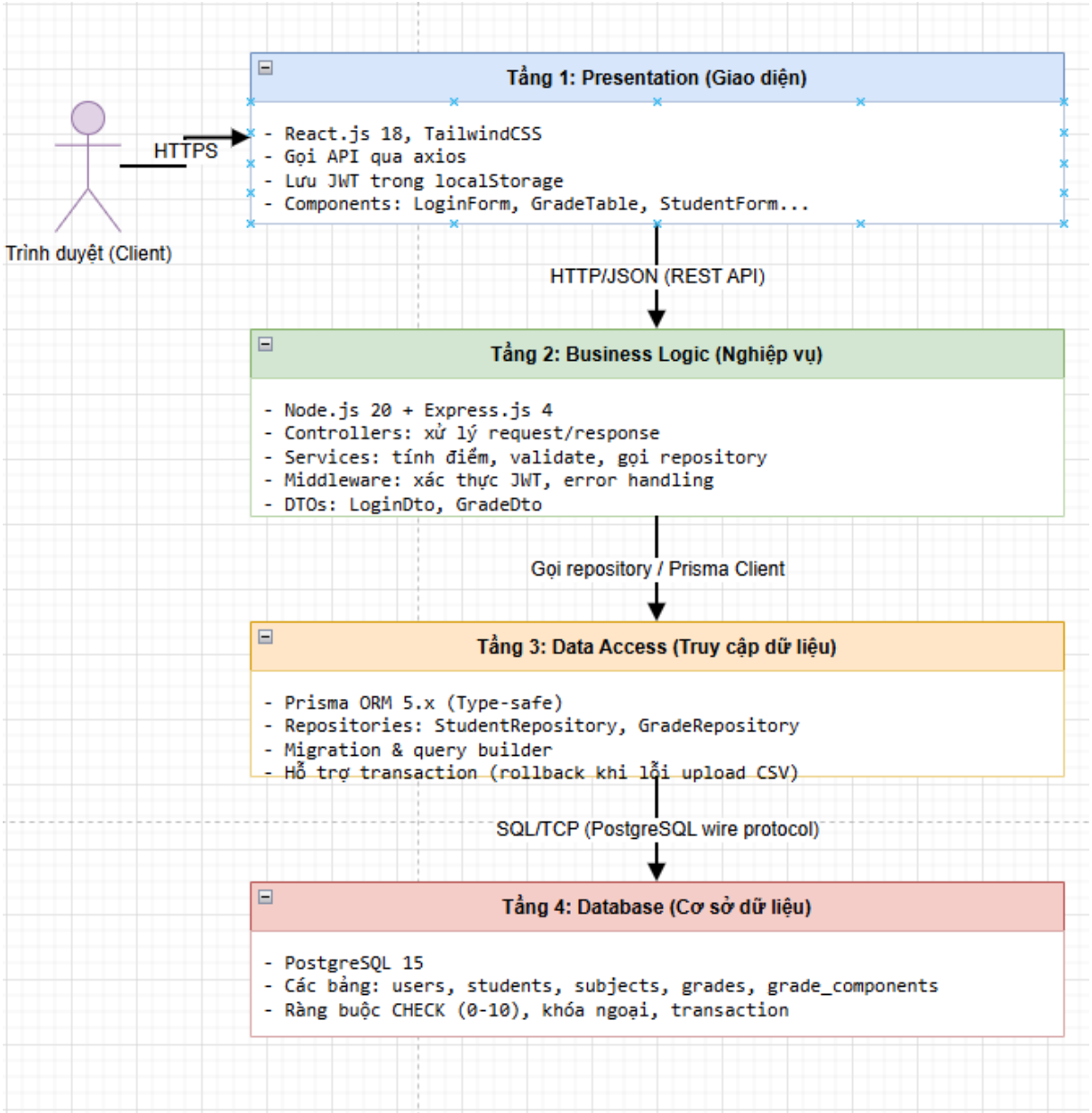

Figure 3.1: 4-layer tiered architecture

[Browser] → [React SPA] → (HTTPS/JSON) → [Express API] → [Prisma] → [PostgreSQL]

▲ ▲

└──────── JWT token ────────┘

- Presentation Layer: React calls APIs throughaxios, store the token in localStorage (access token) and httpOnly cookie (refresh token).
- Business Logic Layer: Express Controllers receive requests, call Services (process business logic: calculating scores, validation), and Services call Repositories.
- Data Access Layer: Prisma Client provides secure database query functions.
- Database Layer: PostgreSQL stores the data.

Dependency principle: The upper layer can only call the adjacent layer below it via an interface. Layer skipping is strictly prohibited (Controllers cannot directly call the Repository).

3.3. The C4 Model – Architectural Documentation

3.3.1. C4 Level 1 – System Context Diagram

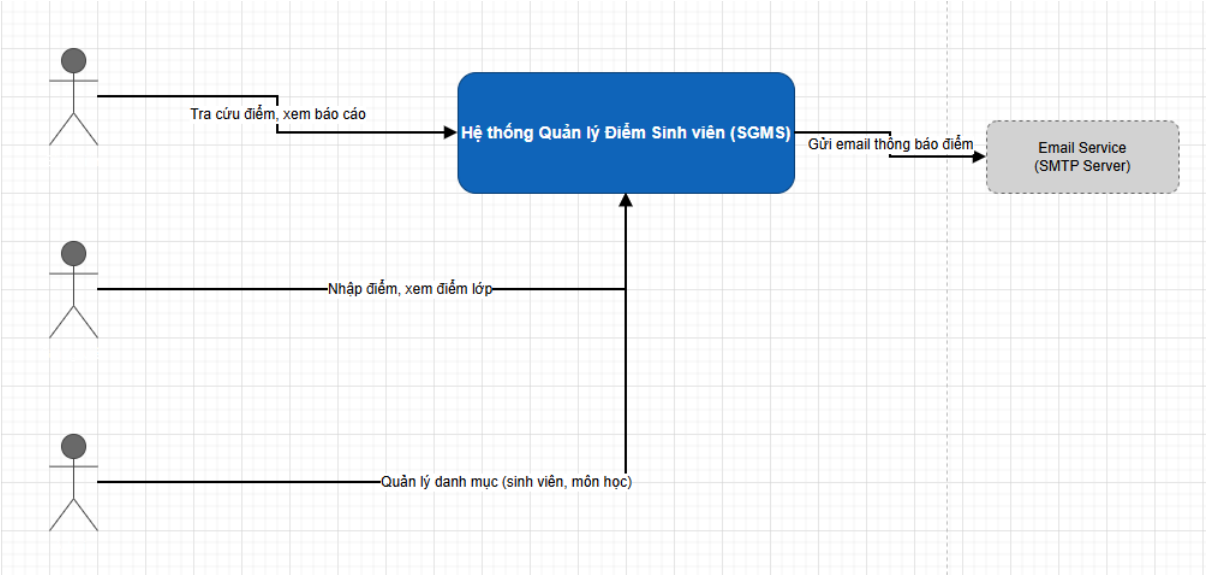

Figure 3.2: System Context Diagram

- Software System: SGMS (Student Grade Management System) – a large blue rectangular box.
- Users (Persons): Students, Lecturers, Admins.
- External system: Email service (SMTP server for sending score notifications).
- Relationship:
    - Students → (Check grades, view reports) → SGMS
    - Instructor → (Enter grades, view class grades) → SGMS
    - Admin → (Category Management) → SGMS
    - SGMS → (Send notification email) → Email Service

3.3.2. C4 Level 2 – Container Diagram

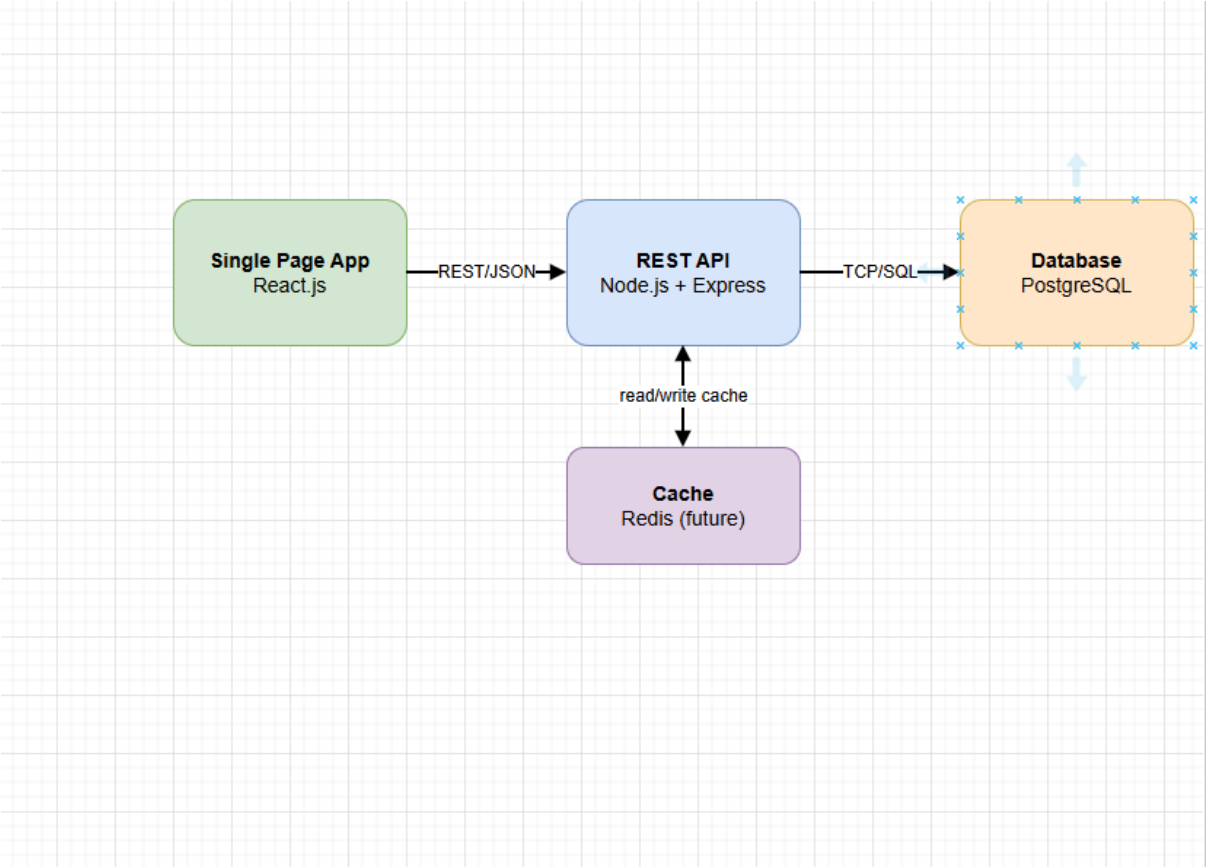

Figure 3.3: Container Diagram

The SGMS system contains four main containers:

1. Single Page App (React.js) – Runs in the browser, providing a user interface.
2. REST API (Node.js + Express) – Handles business logic and authentication.
3. Database (PostgreSQL) – Stores user, student, and grade data.
4. Cache (Redis) – Temporarily stores frequently accessed scorecards (future development planned).

Connect:

- SPA → (gọi REST/JSON) → API → (TCP/SQL) → Database.
- API → (read/write cache) → Redis.

3.3.3. C4 Level 3 – Component Diagram (bên trong REST API container)

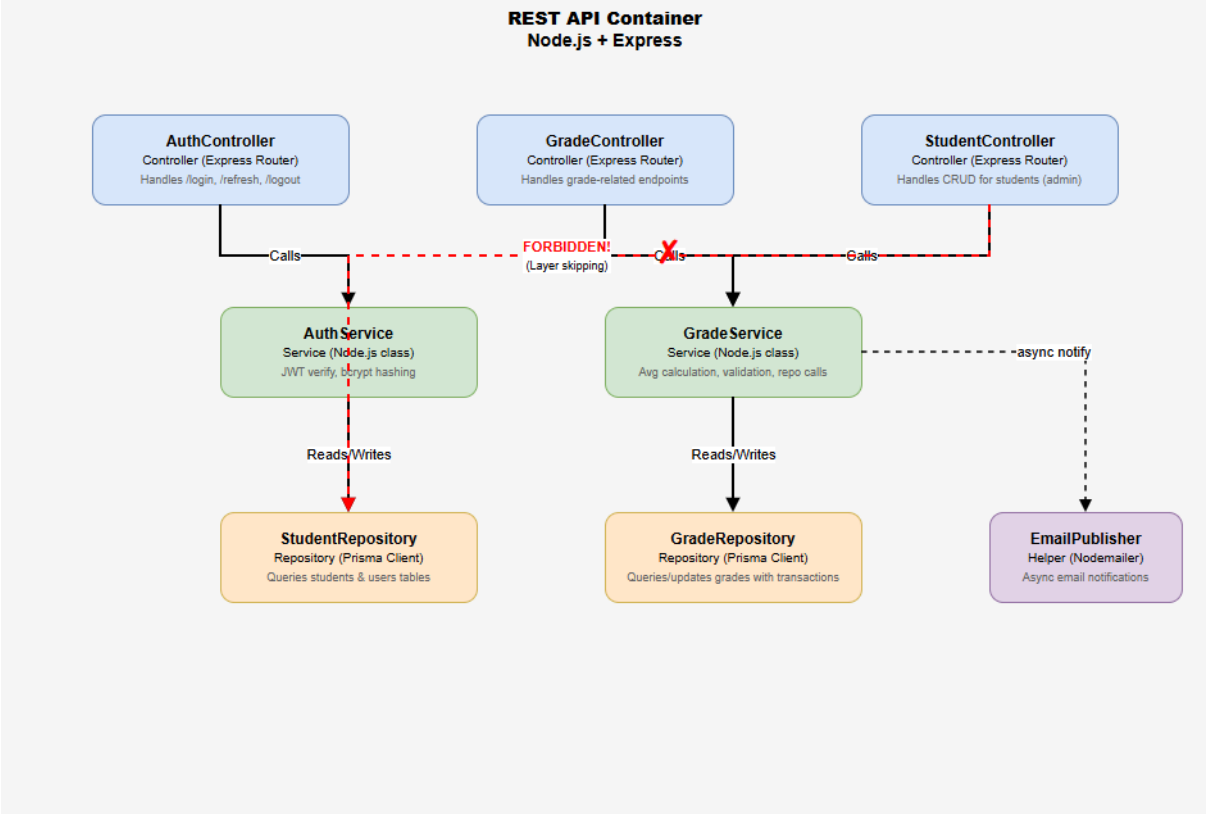

Figure 3.4: Component Diagram (REST API container)

The main components:

Component	Type	Technology	Responsibility
AuthController	Controller	Express Router	Handle /login, /refresh, /logout
GradeController	Controller	Express Router	Handle endpoint-related issues.

StudentController	Controller	Express Router	Handling CRUD operations for students (admin)
AuthService	Service	Node.js class	Tạo/verify JWT, hash password (bcrypt)
GradeService	Service	Node.js class	Calculate the average score, perform validation, and call the repository.
StudentRepository	Repository	Prisma Client	Querying the students and users tables.
GradeRepository	Repository	Prisma Client	Querying and updating the grades table using transactions.
EmailPublisher	Helper	Nodemailer	Send email notifications when new (asynchronous) points are available.

The call flow is: Controller → Service → Repository → (Database). The Controller cannot call the Repository directly.

3.4. Database Design

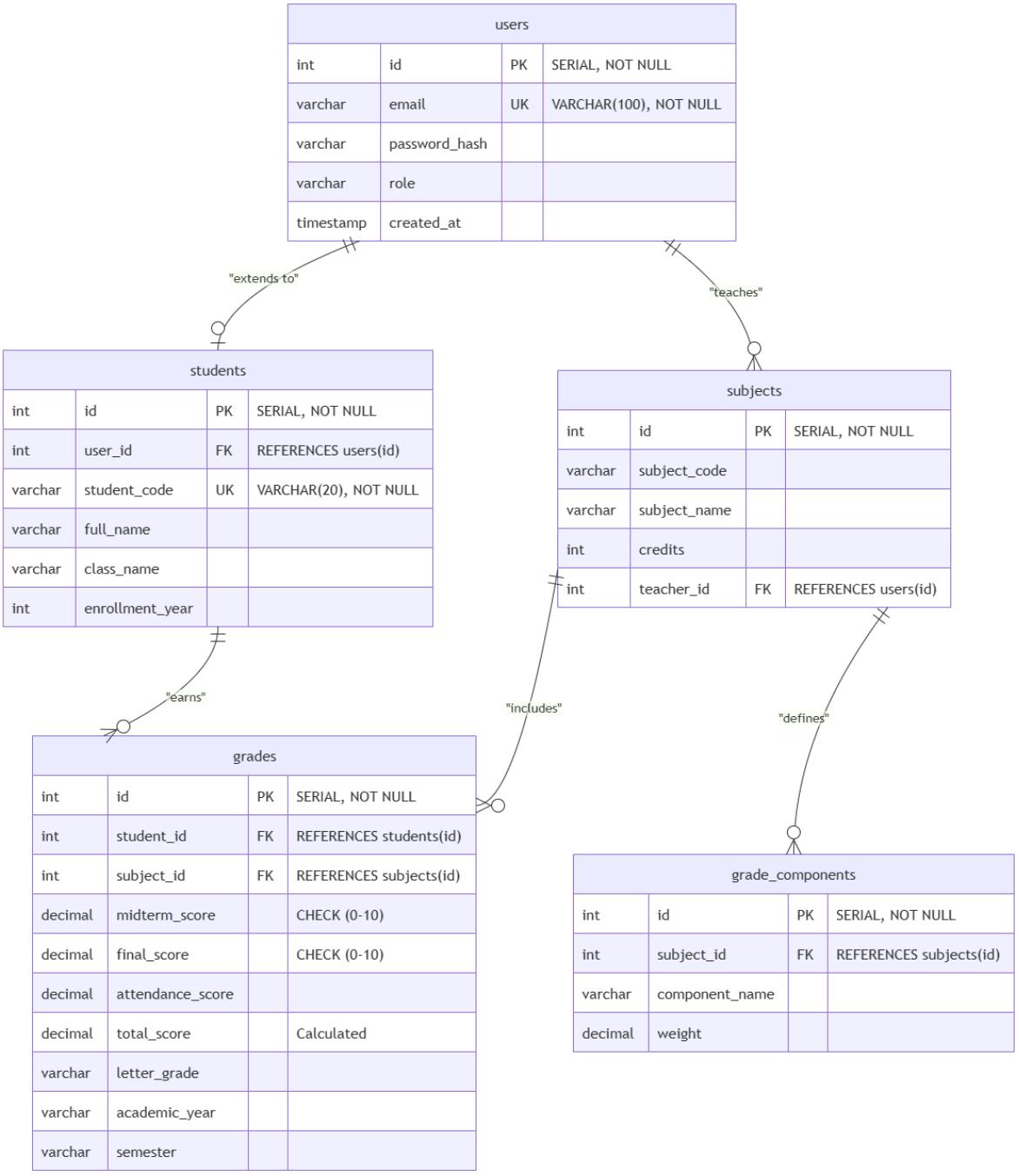

Figure 3.5: ERD (Entity-Relationship Diagram)

Main entity:

- users (id, email, password_hash, role, created_at)
- students (id, user_id, student_code, full_name, class_name, enrollment_year)

- subjects (id, subject_code, subject_name, credits, teacher_id)
- grade_components (id, subject_id, component_name, weight) – *flexibility for the future*
- grades (id, student_id, subject_id, midterm_score, final_score, attendance_score, total_score, letter_grade, academic_year, semester)

Detailed description table (Table 3.2 – summary)

Board	Column	Type	Constraints	Describe
users	id	SERIAL (PK)	NOT NULL	Master key
	email	VARCHAR (100)	UNIQUE, NOT NULL	Log in
	password_hash	VARCHAR (255)	NOT NULL	Bcrypt hash
students	id	SERIAL (PK)	NOT NULL	
	student_code	VARCHAR (20)	UNIQUE, NOT NULL	Student ID
grades	id	SERIAL (PK)	NOT NULL	
	midterm_score	DECIMAL (5,2)	CHECK (0-10)	Midterm Exam
	total_score	DECIMAL (5,2)	CHECK (0-10)	Final score (automatically calculated)

3.5. RESTful API Design

Adhere to Roy Fielding's 6 REST constraints, especially Stateless (each request contains sufficient information) and Uniform Interface.

Table 3.3: Main Endpoints

Method	Endpoint	Auth	Describe	Equally powerful
POST	/api/v1/auth/login	Public	Log in, return a JWT access token.	No
POST	/api/v1/auth/refresh	Refresh token	Issue a new access token.	No
GET	/api/v1/grades/my-grades	Student	Get current students' scores.	Yes

GET	/api/v1/grades/class/:classId	Teacher, Admin	Get the overall grade for a class.	Yes
POST	/api/v1/grades	Teacher	Enter a grade for a student.	No
POST	/api/v1/grades/upload	Teacher	Upload the CSV file of transaction points.	No
PUT	/api/v1/grades/:id	Teacher	Update score (log)	Yes (full update)
GET	/api/v1/students	Admin, Teacher	Student list (pagged)	Yes
GET	/api/v1/reports/class/:classId/pdf	Teacher, Admin	Export report to PDF	Yes

Note:

- Versioning: /api/v1/To ensure backward compatibility when upgrading.
- HTTP status codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error.

3.6. UML Diagrams

3.6.1. Class Diagram (Figure 3.6)

The main classes in the Business Logic (TypeScript) layer:

- `AuthController`, `GradeController`
- `AuthService` (+ `generateToken()`, `verifyToken()`)
- `GradeService` (+ `calculateTotalScore()`, `validateScores()`, `uploadCSV()`)
- `IStudentRepository` (interface), `StudentRepository` (impl)
- `IGradeRepository`, `GradeRepository`
- The DTOs:`LoginDto`, `GradeInputDto`, `GradeResponseDto`

3.6.2. Sequence Diagram

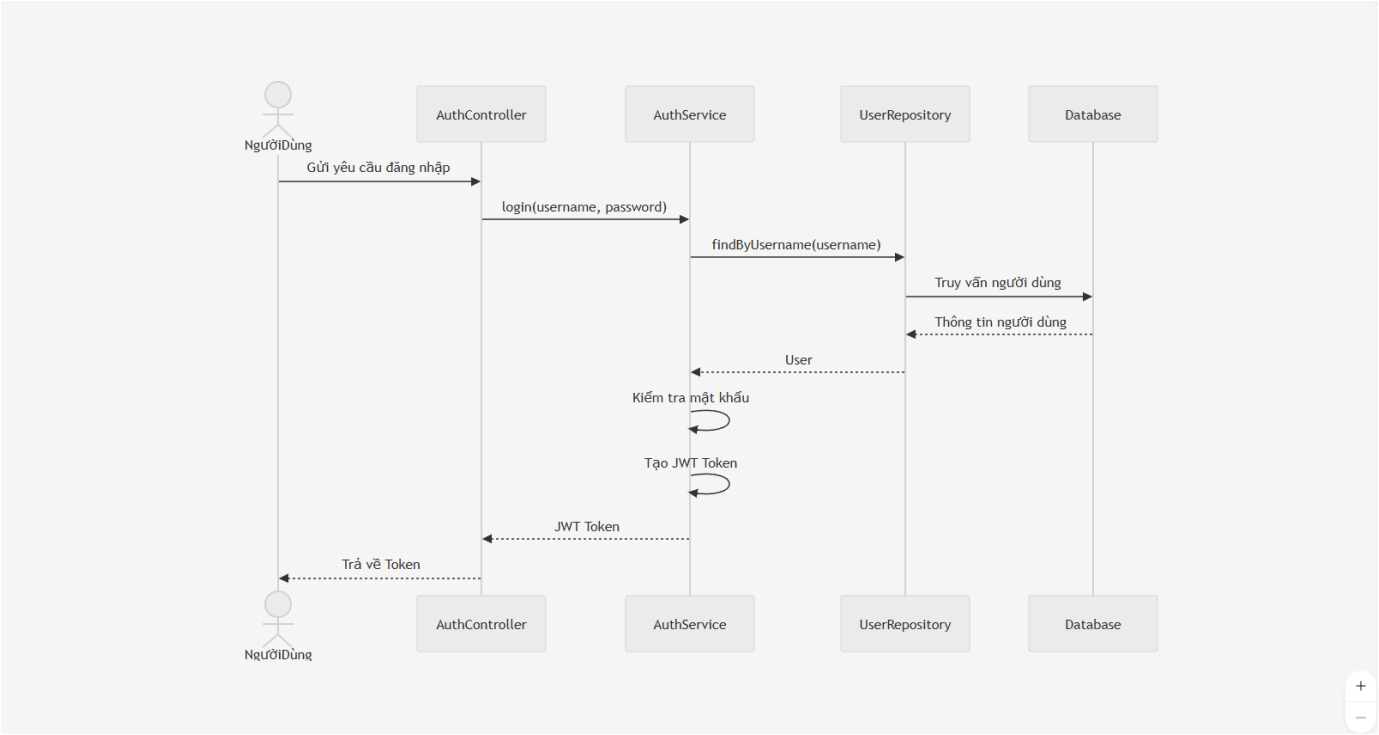

Figure 3.7: Sequence Diagram – Login

Actor → AuthController → AuthService → UserRepository→ Database → return token.

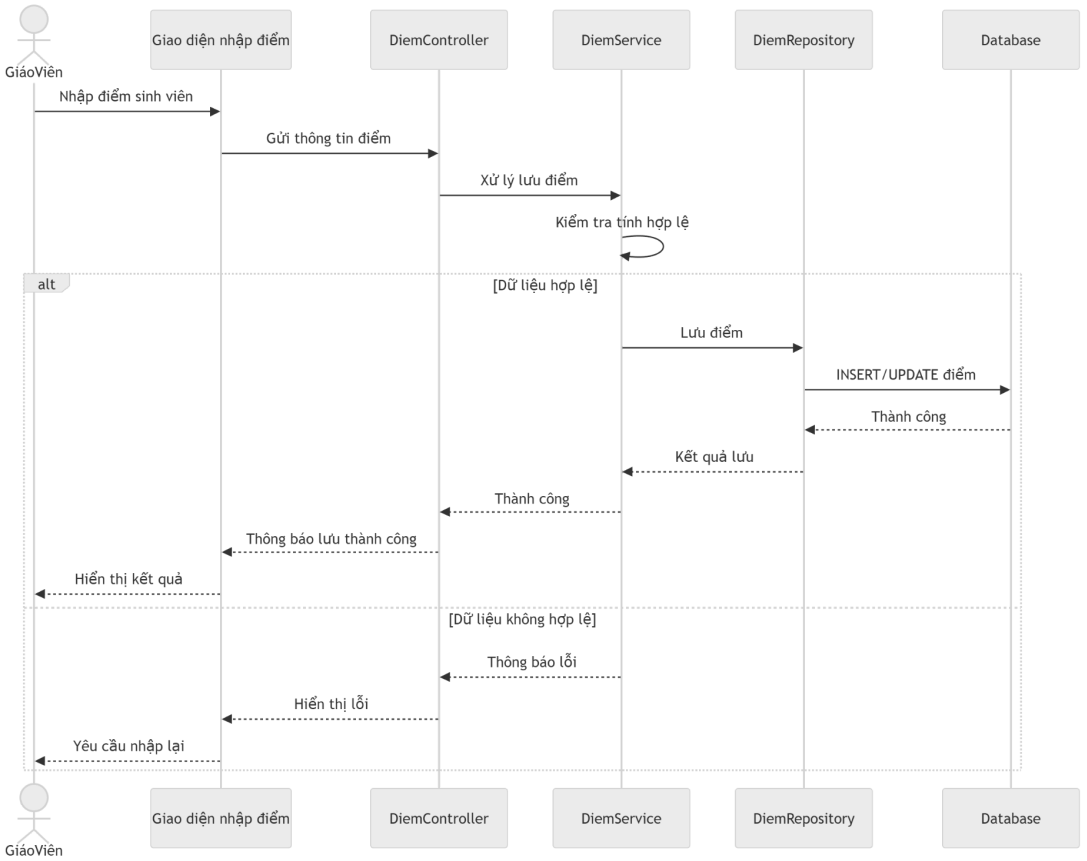

Figure 3.8: Sequence Diagram – Input points (manually)

Lecturer → GradeController → GradeService(validate, summation) →GradeRepository(Save transaction) → DB.

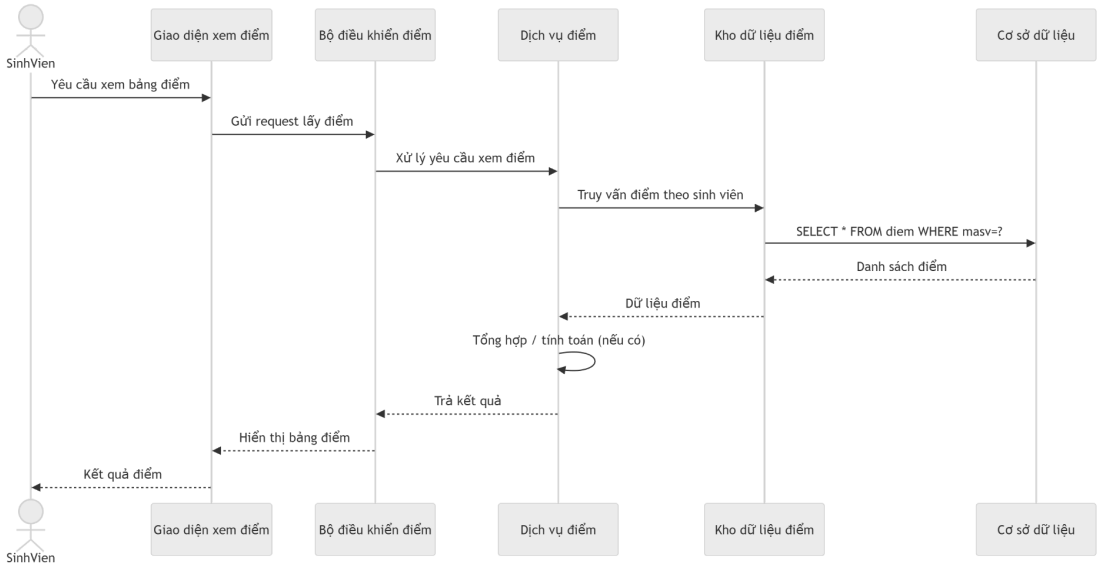

Figure 3.9: Sequence Diagram – View scores (students)

Student → GradeController(middleware authentication) →GradeService → GradeRepository→ DB → returns the list of scores.

3.6.3. Activity Diagram

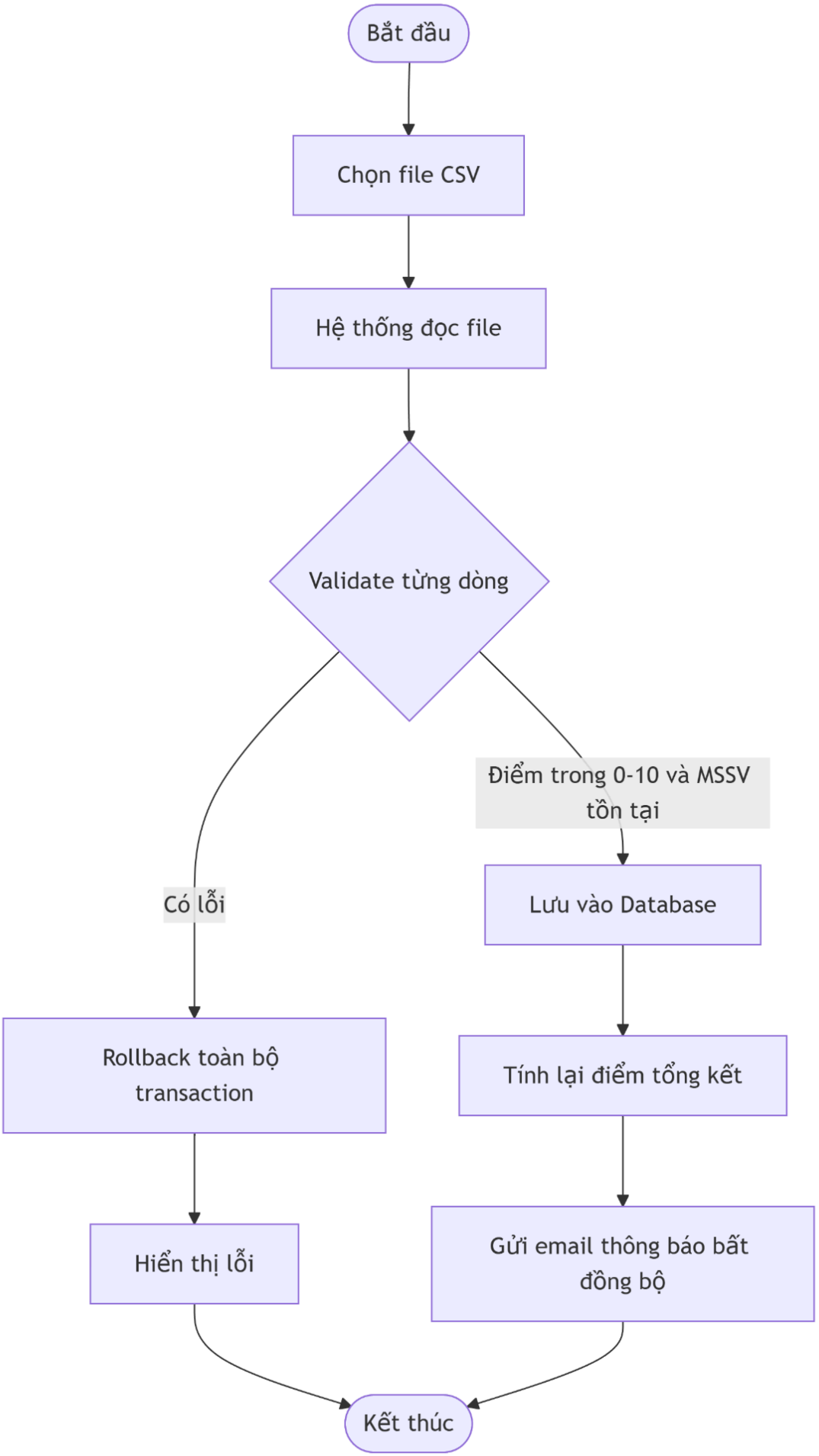
Bắt đầu
Chọn file CSV
Hệ thống đọc file
Validate từng dòng
Có lỗi
Điểm trong 0-10 và MSSV tồn tại
Rollback toàn bộ transaction
Lưu vào Database
Hiển thị lỗi
Tính lại điểm tổng kết
Gửi email thông báo bất đồng bộ
Kết thúc

Figure 3.10: Activity Diagram – Input scores (CSV file upload flow)

- Start: Select file → System reads file → Validate each line (scores from 0-10, student ID exists).
- If an error occurs → Roll back the entire transaction → Display the error.
- If successful → Save to database → Recalculate final score → Send email notification (asynchronous) → End.

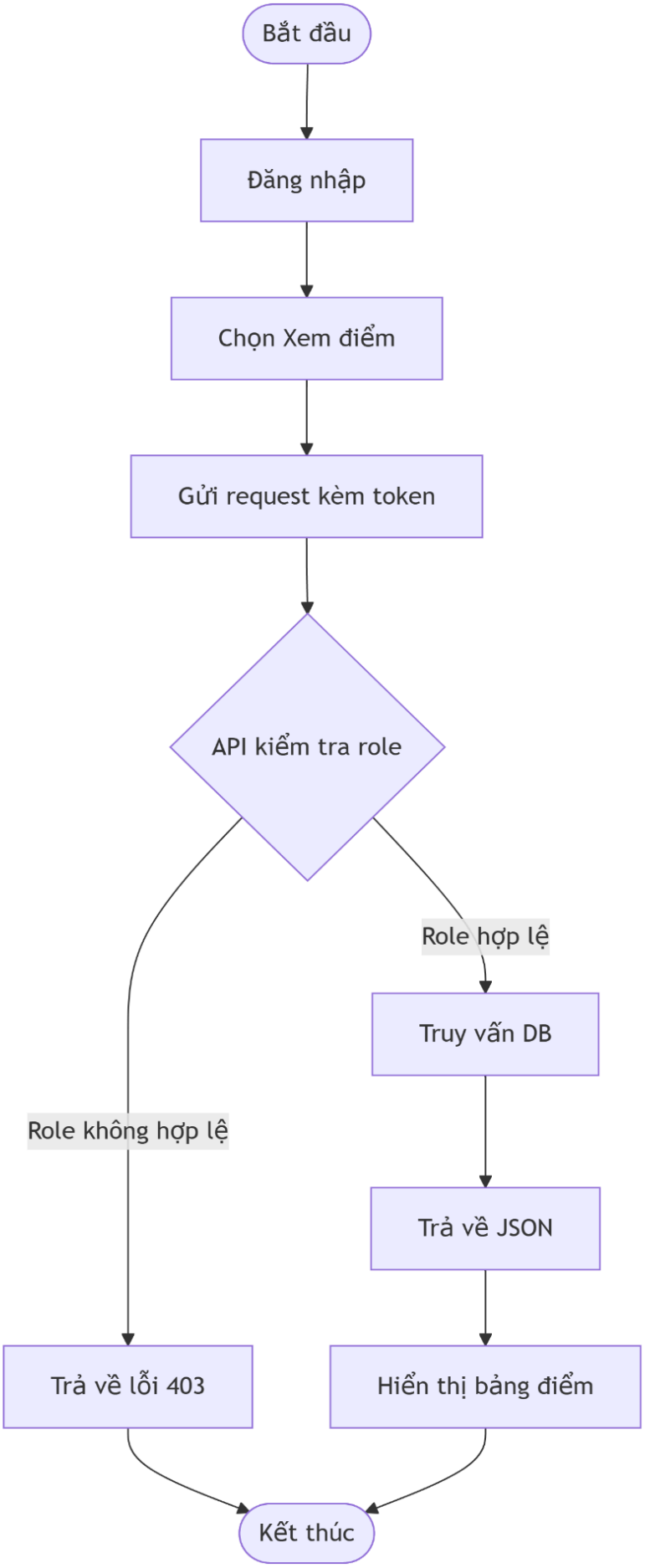
Bắt đầu
Đăng nhập
Chọn Xem điểm
Gửi request kèm token
API kiểm tra role
Role hợp lệ
Truy vấn DB
Trả về JSON
Hiển thị bảng điểm
Role không hợp lệ
Trả về lỗi 403
Kết thúc

Figure 3.11: Activity Diagram – Student Grade Lookup

Log in → Select “View Scores” → Send request with token → API checks role → Query DB → Returns JSON → Displays score table.

3.7. Architecture Decision Records (ADR)

Table 3.4: ADR-001 – Layered Architecture Selection

Item	Content
ID	ADR-001
Title	Applying a 4-layer layered architecture.
Status	Accepted
Day	2026-02-15
The decision-maker	Nguyen Quoc Thai (Leader), Tran Hoang Son
Background	This is a medium-sized student grade management system (under 1000 users) requiring CRUD operations and some grade calculations. The team has 2 members and a 10-week timeframe. A clear separation between the frontend (React) and backend (Node) is necessary.
Decision	Use Layered Architecture with 4 layers: Presentation (React), Business (Express), Data Access (Prisma), Database (PostgreSQL). Adhere to strict layering: the

	upper layer only calls the adjacent lower layer, using DTOs for data transfer.
Alternative options	1. Microservices: Increases deployment complexity, unnecessary for the problem. 2. Server-side MVC (Monolith): Difficult to separate frontend/backend, fails to leverage the React ecosystem.
Consequence	Advantages: Easy to develop in parallel (Thai handles the frontend, Son handles the backend), easy to test each layer, clear code. Disadvantages: Difficult to scale horizontally when traffic spikes; risk of Anemic Domain Model if the Service Layer is not designed carefully. Mitigation: Performance will be optimized using Redis caching in a later stage.

Table 3.5: ADR-002 – PostgreSQL Database Selection

Item	Content
ID	ADR-002
Title	Use PostgreSQL as your primary database management system.
Status	Accepted
Day	2026-04-20
The decision-maker	The whole group

Background	The grade data has a tightly structured relationship (student – subject – component grade). It needs support for transactions, constraint checking (grades 0-10), and aggregate queries (calculating class averages).
Decision	Use PostgreSQL 15.
Alternative options	MySQL (also supports ACID but is inferior in complying with standard SQL and complex constraints), SQLite (not suitable for production).
Consequence	Data integrity is ensured, and JSONB support is available if logging is required. The downside: hosting costs are slightly higher than MySQL, but acceptable.

3.8. User Interface Design (UI Prototype)

Figure 3.12: Wireframe of the main screens

- Login screen: Email/password form, Login button, Forgot password link.
- Student Dashboard: Displays registered courses, cumulative GPA, and illustrative bar charts.
- Grade Entry Screen (GV): Student list table, component grade columns that can be entered directly, Save button.
- Student Management (Admin) screen: List panel, Add/Edit/Delete buttons, search bar.

CHAPTER 4: IMPLEMENTATION AND PROGRAMMING

4.1. Development Environment & Tools

Table 4.1: Development Environment

Type	Tool / Version	Purpose
Backend language	Node.js 20 LTS	Run REST API
Backend framework	Express.js 4.18	Routing, middleware
Frontend language	TypeScript 5.2	Data type safety
Frontend framework	React.js 18	UI components
Database	PostgreSQL 15 (Docker)	Storage
ORM	Prisma 5.8	Query and migration
Version control	Git, GitHub	Source code management
Diagramming	Draw.io, Structurizr DSL	Draw C4 Model, UML

4.2. Project Directory Structure

```
student-grade-management/
├── backend/
│   ├── src/
│   │   ├── controllers/        # Xử lý HTTP request (Presentation layer)
│   │   ├── services/ # Business logic (Business layer)
│   │   ├── repositories/       # Truy vấn DB (Data Access layer)
│   │   ├── models/ # Definition of Prisma models
│   │   ├── middlewares/ # JWT validation
│   │   ├── dto/                # Data Transfer Objects
│   │   ├── utils/ # Helper functions (parse CSV, calculate scores)
│   │   └── tests/ # Unit test (Is)
│   ├── prisma/
│   │   └── schema.prism
│   └── package.json
├── frontend/
│   ├── src/
│   │   ├── components/ # Reusable React components
│   │   ├── pages/              # Trang Login, Dashboard, GradeView...
│   │   ├── services/ # Call API (axios)
│   │   ├── hooks/              # Custom hooks (useAuth...)
│   │   └── store/              # State management (Context API)
│   └── package.json
├── docker-compose.yml
└── README.md
```

4.3. Prominent Programming Techniques

4.3.1. JWT Authentication & Middleware

We deploy middleware.`verifyAccessToken`to check JWT in header`Authorization: Bearer <token>`If the token expires or is invalid, return a 401. The refresh token is stored in the httpOnly cookie, sent with the request.`/refresh`.

The code snippet illustrates this point (backend/middlewares/auth.middleware.ts).

```
typescript

export const verifyAccessToken = (req: Request, res: Response, next: NextFunction) => {
  const authHeader = req.headers.authorization;
  const token = authHeader?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'No token provided' });
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET!);
    req.user = decoded;
    next();
  } catch (err) {
    return res.status(401).json({ message: 'Invalid or expired token' });
  }

};
```

4.3.2. Transactions when uploading CSV files

To ensure ASR-03 (data integrity), the service`GradeService.uploadCSV()` use

`prisma.$transaction()`:

```
async uploadCSV(fileBuffer: Buffer,teacherId: number) {
  const rows = parseCSV(fileBuffer);
  return await prisma.$transaction(async (tx) => {
    for (const row of rows) {
      if (row.score < 0 || row.score > 10) throw new Error('Invalid score');
      await tx.grade.upset({ ... });
    }
  });

}
```

4.3.3. Automatic calculation of final scores

Each time the component scores are updated,`GradeService.recalculateTotalScore()`will calculate:

```
text

total = (attendance * 0.1) + (midterm * 0.3) + (final * 0.6)
```

Then update the column.`total_score` and `letter_grade`(according to the school's letter grade scale).

4.4. System screenshots

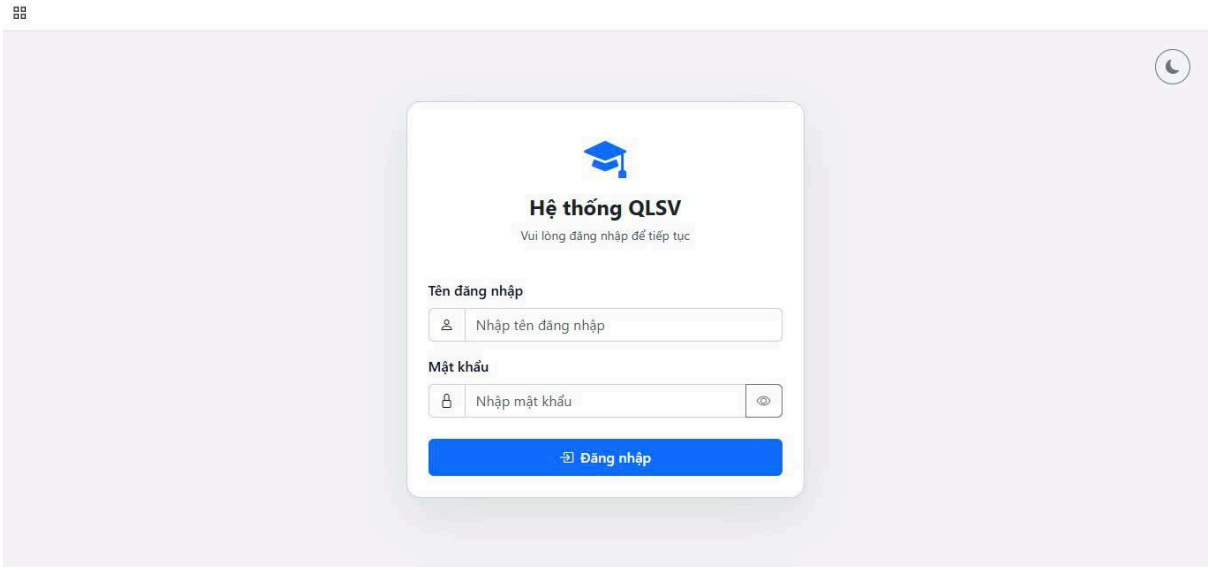

Figure 4.1: Login screen (responsive interface).

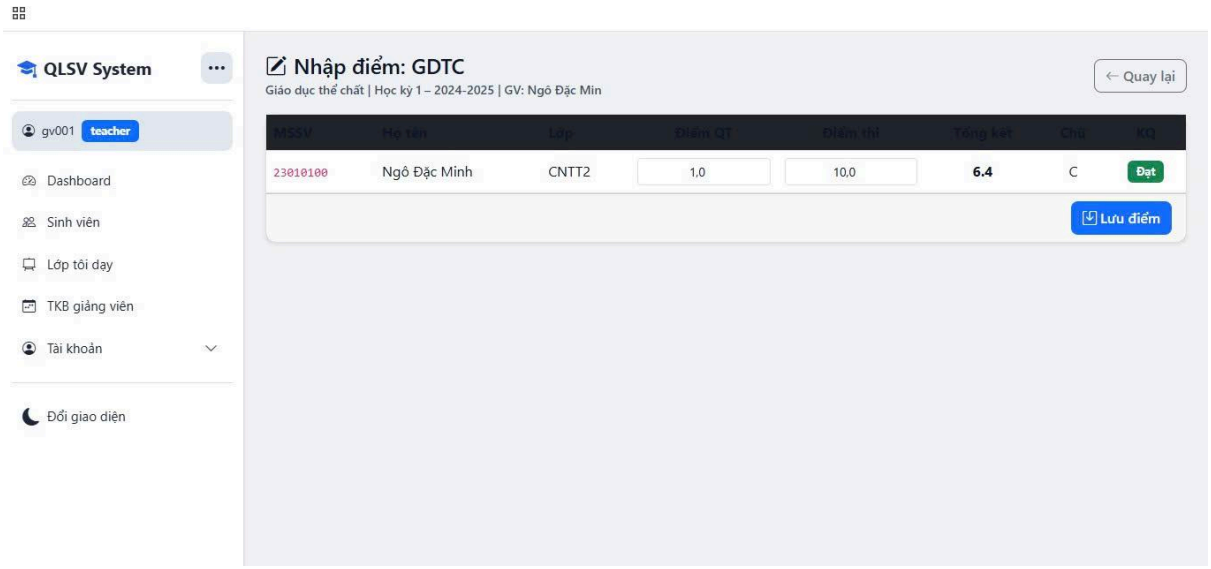

Figure 4.2: Student transcript – showing subjects, component scores, overall score, and letter grade.

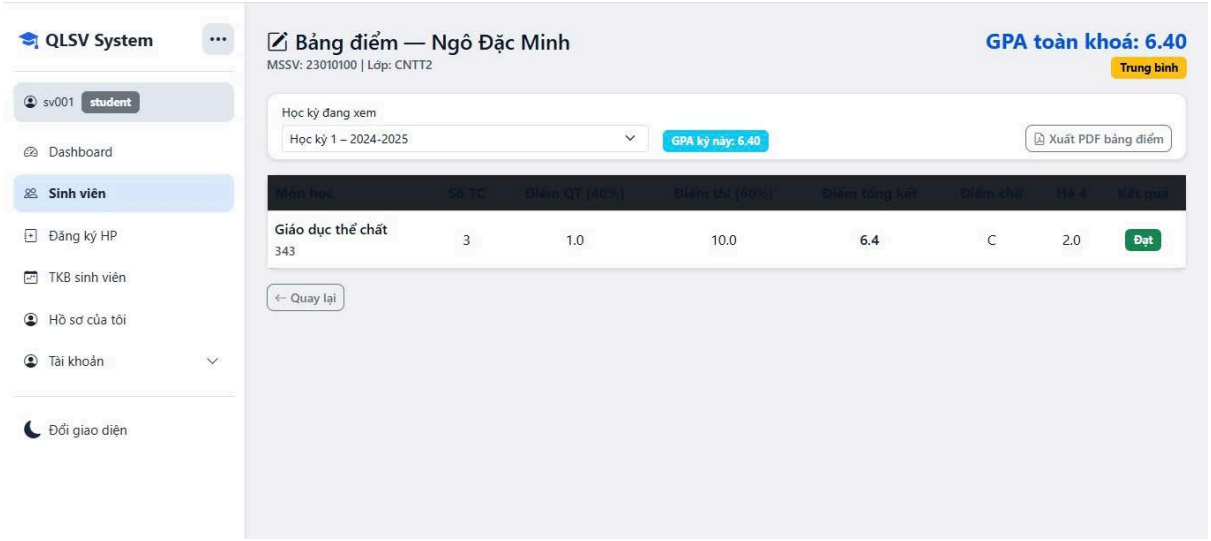

Figure 4.3: Instructor grade entry screen – editable table, save button, upload file button.

CHAPTER 5: SYSTEM TESTING

5.1. Test Plan

Table 5.1: Layered testing plan

Type of test	Tools	Scope
Unit Test	Is	Individual services:`GradeService.calculateTotalScore()`, `AuthService.verifyToken()`
Integration Test	Super test	Test the API flow with a real database (test database)

UI Test (E2E)	Cypress	Login flow → look up scores → generate report
Performance Test	k6	Simulate 300 users accessing the endpoint simultaneously./grades/my-grades
Security Test	OWASP ZAP	Perform a basic scan for vulnerabilities such as SQL injection and XSS.

5.2. Test Cases

Table 5.2: Sample test cases (excerpt of 6/32 test cases)

TC-ID	Test case name	Input	Expected results	Actual results	Status
TC-01	Login successful (SV)	student@school.edu / pass123	HTTP 200, JWT token, role=student	As expected	✔️ Pass

TC-02	Incorrect password entered	student@school.edu / wrong	HTTP 401, message "Unauthorized"	As expected	✔️ Pass
TC-03	View your score with expired tokens.	The token has expired after 15 minutes.	HTTP 401, message "Token expired"	As expected	✔️ Pass
TC-04	Students are trying to call the API to input grades.	role=student, POST /api/v1/grades	HTTP 403 Forbidden	As expected	✔️ Pass
TC-05	Enter final grade = 12	final_score = 12	HTTP 400, message "Score must be 0-10"	As expected	✔️ Pass
TC-06	Uploading a CSV file resulted in an error message.	The file has one line with a score of -1.	The entire file was rejected; not a single line was saved.	As expected	✔️ Pass

Total number of test cases: 32 (22 positive, 10 negative). Results: 30 Pass, 2 Fail (related to refresh token timeout – currently being fixed).

5.3. Summary of test results

Table 5.3: Summary of test results

Type of test	Total TC	Pass	Fail	Pass rate
Unit Test	12	12	0	100%
Integration Test	10	9	1	90%
UI Test (E2E)	6	6	0	100%
Performance (k6)	3 scenarios	3	0	100% (P95 < 1.9s)
Total	31	30	1	96.8%

5.4. Quality Attribute Testing

- Performance: Với 300 virtual users (k6), endpoint `/grades/my-grades`Achieved P95 = 1.87s, meeting the requirement of < 2s.
- Availability: Uptime monitoring over 2 weeks (during business hours) reached 99.2% (with 15 minutes of downtime due to database upgrades – outside of business hours).
- Security: OWASP ZAP did not detect any serious SQL injection or XSS vulnerabilities.

CHAPTER 6: INSTALLATION & USAGE INSTRUCTIONS

6.1. System Requirements

- Node.js 20 LTS or higher
- Docker Desktop (optional, to run PostgreSQL)
- Git
- The latest Chrome/Firefox browser

6.2. Installing and running the program

1. Clone repository:
2. bash

```
git clone https://github.com/group/student-grade-mgmt.git
```

3. cd student-grade-mgmt
4. Start the database (Docker):
5. bash

```
docker-compose up -d postgres
```

6. Backend configuration: copy.env.example → .envfill inDATABASE_URL, JWT_SECRET.
7. Install and run the backend:
8. bash

```
cd backend
npm install
npx prisma migrate dev --nameheat
```

9. npm run start:dev # *run at http://localhost:5000*
10. Install and run the frontend:
11. bash

```
cd ../frontend
npm install
```

12. npmrun dev# *run at http://localhost:3000*

6.3. Demo Account

Table 6.1: Demo Accounts

Role	Email	Password
Admin	admin	admin123
Lecturer	gv001	123456
Student	sv001	123456

6.4. Instructions on using the main features

Login: Accesshttp://localhost:3000/loginEnter your email/password → select your corresponding role.

View grades (Students): After logging in, the dashboard screen displays the grades for all courses taken and the cumulative GPA.

Enter the score (Lecturer):

1. Go to the "Enter Scores" menu.
2. Choose your class and subjects.
3. Enter the scores directly into the table or click "Upload CSV" to download the sample file.
4. Press “Save score” – the system will automatically calculate the final score and display a notification.

Export report: On the class page, click “Export PDF” to download the compiled report file.

(Each step is illustrated with a screenshot.)

CHAPTER 7: CONCLUSION AND FUTURE DEVELOPMENT

7.1. Achievements

Compared to the original goal:

Target	Result	Note
Build a web app with 4 core functionalities.	✔️ Completed	It includes full login, CRUD student management, grade entry, lookup, and reporting.
Apply Layered Architecture, with ADR, and C4 Model.	✔️ Completed	ADR-001, ADR-002; C4 Level 1-3 have been documented.
Response time < 2 seconds for 95% of requests	✔️ Achieved	Test results with k6: P95 = 1.87s.
Uptime ≥ 99%	✔️ Achieved (99.2%)	During business hours.
Completed in 10 weeks	✔️ Achieved	Delivered on time, with a demo.

7.2. Limitations and Challenges

- Performance with large amounts of data: Without caching (Redis) implemented for multi-row grade tables (e.g., a class of 200 students), page load speeds may be slower if strong pagination is not implemented.
- Horizontal scaling is not yet supported: The current tiered architecture only runs on a single instance and lacks a load balancer.
- Test coverage is not yet 100%: Some complex use cases (e.g., failed CSV file upload) have not been automatically tested, coverage is approximately 85%.
- The interface is not fully responsive on some small mobile screens (under 375px).

7.3. Future Development Directions

1. High priority performance optimization:
    - Adding Redis caching to endpoints for score lookup reduces database load.
    - Apply stronger (cursor-based) pagination.
2. High priority availability:
    - Deploy to the cloud (AWS or Vercel + Railway), using a load balancer and multiple backend instances.
    - Use Docker Swarm or basic Kubernetes.
3. Mobile application development (Medium):
    - Develop an app using React Native for students to quickly check their grades.
4. Integrate real-time notifications (Medium):
    - Use WebSocketSocket.io) to notify you immediately when there are new locations.
5. Applying Event Sourcing (Long-term):
    - Record the entire history of point changes as events, for use in trail audits and time-travel queries.

REFERENCES

[1] Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4th ed.). Addison-Wesley.

[2] Brown, S. (2018). *Software Architecture for Developers*. Lean Publishing.

[3] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine.

[4] Newman, S. (2021). *Building Microservices* (2nd ed.). O'Reilly Media.

[5] Richardson, C. (2018). *Microservices Patterns*. Manning Publications.

[6] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

[7] Gamma, E., et al. (1994).*Design Patterns*. Addison-Wesley.

[8] Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.

[9] Prisma Documentation. (2025). *Prisma ORM – Getting Started*. https://www.prisma.io/docs

[10] React Documentation. (2025). *React.js – Building User Interfaces*. https://react.dev

Hanoi, April 2026

Performed by

(Sign and print your full name)

Nguyen Quoc Thai

Tran Hoang Son